Lab Manual

Information and Network Security (INS)

Subject code

……..

5th Semester

Index

Sr No.	Practicals	Date of Start & End	Page no.	Remark	Sign
1	**Implement Caesar cipher encryption- decryption.**				
2	**Implement Monoalphabetic cipher encryption- decryption.**				
3	**Implement playfair cipher encryption- decryption.**				
4	**Implement polyalphabetic cipher encryption- decryption.**				
5	**Implement Hill cipher encryption- decryption.**				
6	**Implement Simple Transposition encryption-decryption**				
7	**Implement One time pad encryption-decryption**				
8	**Implement Diffi-Hellmen key Exchange method.**				
9	**Implement RSA encryption-decryption algorithm.**				
10	**Demonstrate working of Digital Signature using Cryp-tool.**				
11	**Perform various encryption-decryption techniques with cryptool.**				

Practical:-1

AIM:- Implement Caesar cipher encryption-decryption.

CODE:-

```
#include<stdio.h>
#include<conio.h>
#include<iostream>
using namespace std;

void  encryption();
void  decryption();
int main()
{
	char choice;
	cout<<"For Encryption Press e"<<endl<<"For Decrption Press d"<<endl;
	cin>>choice;

	switch(choice)
	{
		case 'e' : encryption();
			break;
		case 'd' : decryption();
			break;
		default :
			cout<<"Invalid Input"<<endl;
	}
}
void encryption()
{
	char mess[100],ch;
	int i,key,tot,k=0;
	cout<<"Enter message to encrypt : "<<endl;
	cin>>mess;
	cout<<"Enter key : "<<endl;
	cin>>key;
	for(i=0;mess[i]!='\0';i++)
	{
		ch=mess[i];
		if(ch>='a' &&ch<='z')
		{
			tot = (int)ch + key;
			//cout<<tot;
			if(tot>122)
			{
				// ch = (((ch - 97) + key) % 26) + 97;
```

```
                        //ch=(ch-'z')+97-1;
                        tot=((tot-122)-1)+97;
                        //cout<<(char)tot;
                }
                mess[i]=(char)tot;
                k=i;
        }
        else if(ch>='A' &&ch<='Z')
        {
                tot = (int)ch + key;
                //cout<<tot;
                if(tot>90)
                {
                        // ch = (((ch - 97) + key) % 26) + 97;
                        //ch=(ch-'z')+97-1;
                        tot=((tot-90)-1)+65;
                        //cout<<(char)tot;
                }
                mess[i]=(char)tot;
                k=i;
        }
    }
    cout<<"Encrypted message is : ";
    for(k=0;mess[k]!='\0';k++)
    {
    cout<<mess[k];
    }
}
void decryption()
{
    char mess[100],ch;
    int i,key,tot,k=0;

    cout<<"Enter message to decrypt : "<<endl;
    cin>>mess;
    cout<<"Enter key : "<<endl;
    cin>>key;
    for(i=0;mess[i]!='\0';i++)
    {
        ch=mess[i];

        if(ch>='a' &&ch<='z')
        {
                tot = (int)ch - key;
                if(tot<97)
                {
```

```
                        tot=((tot+122)+1)-97;
                        //cout<<(char)tot;
                }
                mess[i]=(char)tot;
                k=i;
        }
        else if(ch>='A' &&ch<='Z')
        {
                tot = (int)ch - key;
                //cout<<tot;
                if(tot<65)
                {
                        tot=((tot+90)+1)-65;
                        //cout<<(char)tot;
                }
                mess[i]=(char)tot;
                k=i;
        }
}
cout<<"Decrypted message is : ";
for(k=0;mess[k]!='\0';k++)
{
cout<<mess[k];
}
}
```

OUTPUT:-

```
For Encryption Press e
For Decrption Press d
e
Enter message to encrypt :
hellomit
Enter key :
16
Encrypted message is : xubbecyj
--------------------------------
Process exited after 23.74 seconds with return value 0
Press any key to continue . . .
```

```
For Encryption Press e
For Decrption Press d
d
Enter message to decrypt :
xubbecyj
Enter key :
16
Decrypted message is : hellomit
--------------------------------
Process exited after 45.76 seconds with return value 0
Press any key to continue . . .
```

Practical:-2

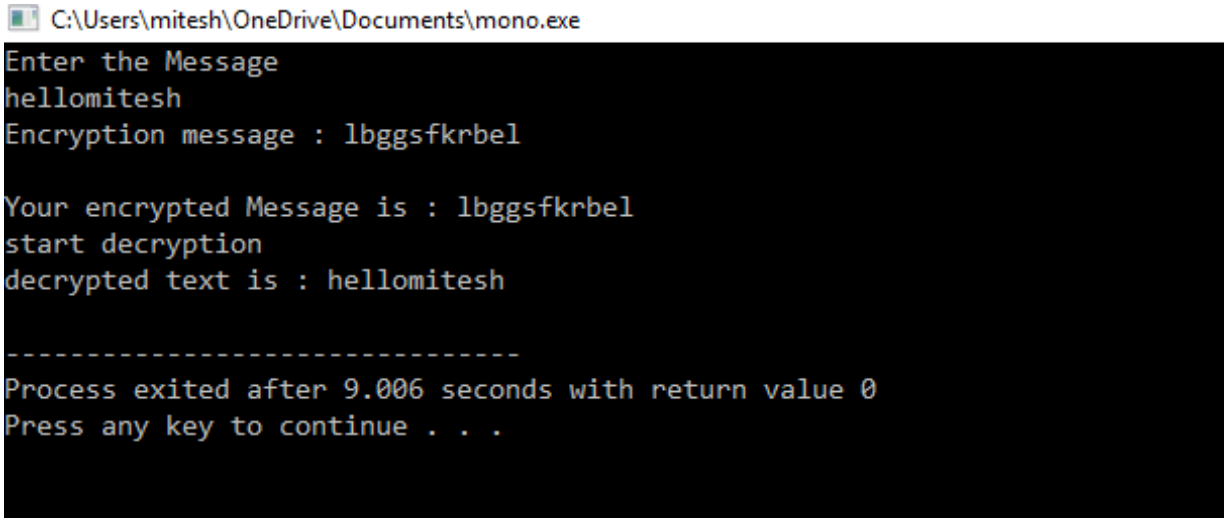
AIM: - Implement Monoalphabetic cipher encryption-

decryption. CODE:-

```
#include<stdio.h>
#include<conio.h>
#include<strings.h>
#include<iostream>
using namespace std;

void encryption();
void decription();

int main()
{
	int i,j;
	char text[200];
	char plain[26] = {'a','b','c','d','e','f','g','h','i','j','k','l','m','n','o','p','q','r','s','t','u','v','w','x','y','z'};
	char key[26] = {'z','x','c','v','b','n','m','l','k','j','h','g','f','d','s','a','q','w','e','r','t','y','u','i','o','p'};

	cout<<"Enter the Message"<<endl;
	cin>>text;
	for(i=0;i<strlen(text);i++)
	{
		for(j=0;j<26;j++)
		{
			if(plain[j]==text[i])
			{
				text[i]=key[j];
				break;
			}
		}
	}
	cout<<"Encryption message : "<<text<<"\n"<<endl;

	cout<<"Your encrypted Message is : "<<text<<endl;
	cout<<"start decryption"<<endl;
	for(i=0;i<strlen(text);i++)
	{
		for(j=0;j<26;j++)
		{
			if(text[i]==key[j])
			{
				text[i]=plain[j];
				break;
```

```
                    }
                }
        }
        cout<<"decrypted text is : "<<text<<endl;

}
```

OUTPUT:-



Practical:-3

AIM: - Implement playfair cipher encryption-

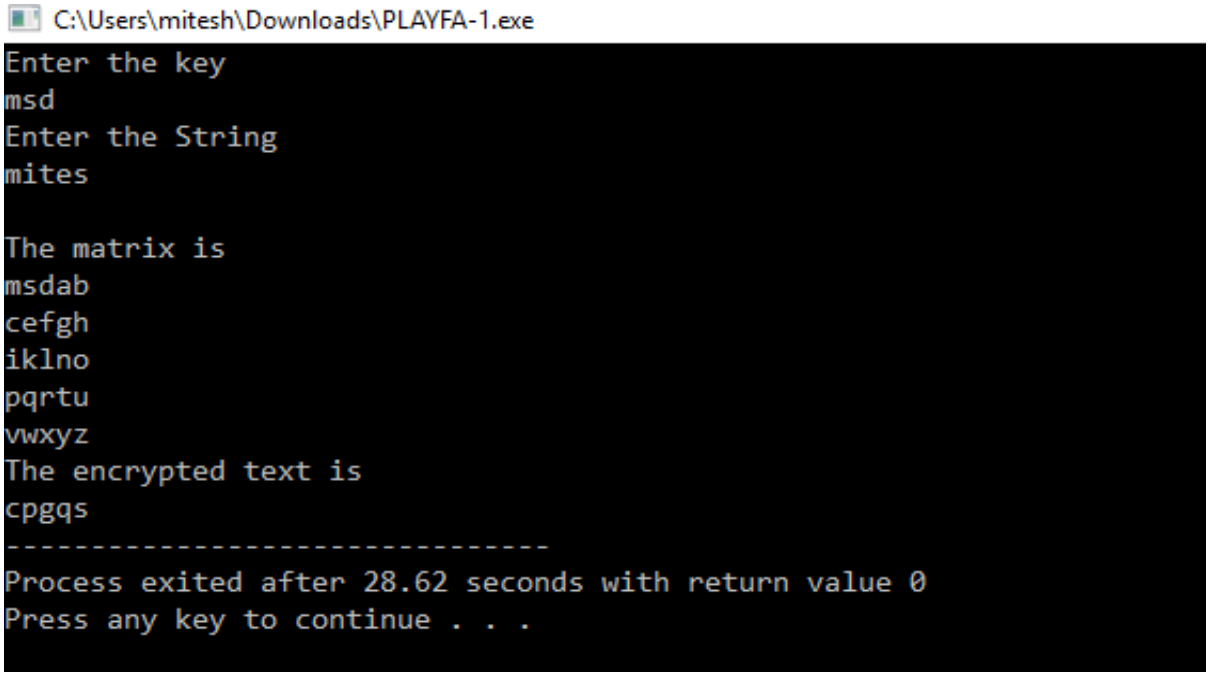
decryption. CODE:-

```
#include<stdio.h>
#include<conio.h>
#include<string.h>
int main()
{
	char v,w,ch,string[100],arr[5][5],key[10],a,b,enc[100];
	int temp,i,j,k,l,r1,r2,c1,c2,t,var;
	printf("Enter the key\n");
	scanf("%s",&key);
		printf("Enter the String\n");
			scanf("%s",&string);
	for (i=0;key[i]!='\0';i++) {
		for (j=i+1;key[j]!='\0';j++) {
			if(key[i]==key[j]) {
				temp=1;
				break;
			}
		}
	}
	if(temp==1)
	printf("invalid key"); else {
		k=0;
		a='a';
		//printf("%c",b);
		for (i=0;i<5;i++) {
			for (j=0;j<5;j++) {
				if(k<strlen(key))
				   arr[i][j]=key[k]; else if(k==strlen(key)) {
					b:
					   for (l=0;l<strlen(key);l++) {
						if(key[l]==a || key[l]=='j') {
							a++;
							goto b;
						}
					}
					arr[i][j]=a;
					if(a=='i')
					   a=a+2; else
					   a++;
				}
				if(k<strlen(key))
```

```
                        k++;
                }
        }
        printf("\n");
        printf("The matrix is\n");
        for (i=0;i<5;i++) {
                for (j=0;j<5;j++) {
                        printf("%c",arr[i][j]);
                }
                printf("\n");
        }
t=0;

        if(strlen(string)%2!=0)
        var=strlen(string)-1;
        for (i=0;i<var;) {
                v=string[i++];
                w=string[i++];
                if(v==w) {
                        enc[t++]=v;
                        enc[t++]='$';
                } else {
                        for (l=0;l<5;l++) {
                                for (k=0;k<5;k++) {
                                        if(arr[l][k]==v||v=='j'&&arr[l][k]=='i') {
                                                r1=l;
                                                c1=k;
                                        }
                                        if(arr[l][k]==w||w=='j'&&arr[l][k]=='i') {
                                                r2=l;
                                                c2=k;
                                        }
                                }
                        }
                        if(c1==c2) {
                                r1++;
                                r2++;
                                if(r1==5||r2==5) {
                                        r1=0;
                                        r2=0;
                                }
                        } else if(r1==r2) {
                                c1++;
                                c2++;
                                if(c1==5||c2==5) {
                                        c1=0;
                                        c2=0;
```

```
                        }
                    } else {
                        temp=r1;
                        r1=r2;
                        r2=temp;
                    }
                    enc[t++]=arr[r1][c1];
                    enc[t++]=arr[r2][c2];
                }
            }
            if(strlen(string)%2!=0)
            enc[t++]=string[var];
            enc[t]='\0';
        }
        printf("The encrypted text is\n");
        for(i=0;i<strlen(enc);i++)
        {
            printf("%c",enc[i]);
        }
        getch();
}
```

OUTPUT:-



Practical:-4

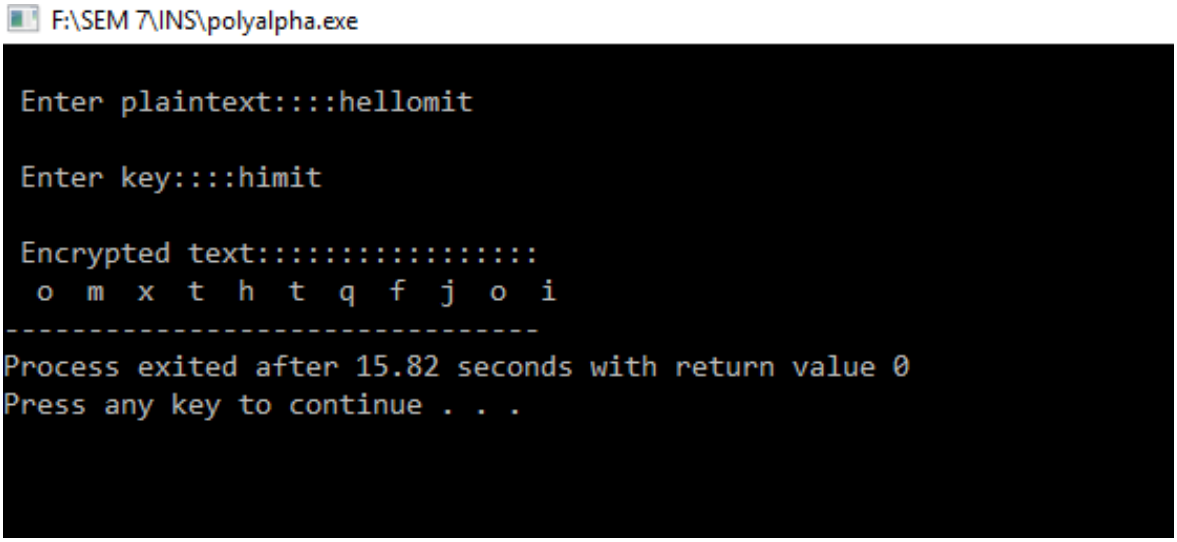
AIM: - Implement polyalphabetic cipher encryption-

decryption. CODE:-

```
#include<stdio.h>
#include<conio.h>
#include<string.h>
int main()
{
int i,j,m,a[26][26],klen,plen;
char p[100],p1[100],e[100],d[100],k[100];
printf("\n Enter plaintext::::");
gets(p);
printf("\n Enter key::::");
gets(k);
for(i=0;i<26;i++)
{
 m=i;
          for(j=0;j<26;j++)
          {
if(m<=25)
                    {
a[i][j]=m+97;
                              m++;
                    }
                   else
                    {
a[i][j]=97;
                              m=1;
                    }
 }
}
plen=strlen(p);
klen=strlen(k);


m=0;
for(i=0;i<plen;i++)
{
 if(p[i]!=32)
          {
p1[m]=p[i];
                    m++;}
}
plen=strlen(p1);
m=0;
```

```
for(i=klen;i<plen;i++)
{
 if(m==klen)
 {
m=0;
 }
 k[i]=k[m];
 m++;
}
/*k[i]='\0';
printf("%s",k);*/

//encryption part
printf("\n Encrypted text:::::::::::::::::\n ");
for(i=0;i<plen;i++)
{
e[i]=a[k[i]-97][p1[i]-97];
printf(" %c ",e[i]);
}
getch();
}
```

OUTPUT:-



Practical:-5

AIM: - Implement Hill cipher encryption-

decryption. CODE:-

```
#include<stdio.h>
#include<conio.h>
#include<string.h>
void main()
{
int i,j,key[5][5],ikey[5][5],row,col,plen,suc;
int devide,count,h,k,no,p1[100],e1[100],d1[100],m,pp[10],temp;
char p[100],e[100],d[100],clen,det;


printf("Enter your plaintext : ");
gets(p);
plen=strlen(p);
printf("\n matrix index : ");
scanf("%d",&suc);
row=suc;
col=suc;
printf("\n ENTER ELEMENTS OF KEY MATRIX (row by row) :\n");
for(i=0;i<row;i++)
for(j=0;j<col;j++)
scanf("%d",&key[i][j]);
devide=(plen+1)/suc;

count=0;
no=0;
for(h=0;h<devide;h++)
{
  for(i=0;i<suc;i++)
  { if(p[count]!='\0')
    {p1[i]=p[count]-97;
    count++;
    }
  }
  for(i=0;i<suc;i++)
  { e1[no]=0;
      for(k=0;k<suc;k++)
      {e1[no]=e1[no]+(key[i][k]*p1[k]);
      }
      no++;
  }
}
```

```
printf("\n PLAIN TEXT:");
puts(p);

printf("\n ENCRYPTED TEXT:");
for(i=0;i<plen;i++)
{e[i]=(e1[i]%26)+97;
printf("%c",e[i]);
}

//decryption part
clen=strlen(e);
devide=(clen+1)/suc;
printf("\n Enter inverse matrix values :");
for(i=0;i<suc;i++)
for(j=0;j<suc;j++)
{scanf("%d",&ikey[i][j]);
}
count=0;
no=0;
for(h=0;h<devide;h++)
{
  for(i=0;i<suc;i++)
  { if(e[count]!='\0')
    {p1[i]=e[count]-97;
    count++;
    }
  }
  for(i=0;i<suc;i++)
  { d1[no]=0;
      for(k=0;k<suc;k++)
      {d1[no]=d1[no]+(ikey[i][k]*p1[k]);
      }
      no++;
  }

}
printf("\n DECRYPTED TEXT:");
for(i=0;i<plen;i++)
{
d[i]=(d1[i]%26)+97;
printf("%c",d[i]);
}
getch();
}
```

OUTPUT:-

F:\SEM 7\INS\hill.exe

```
Enter your plaintext : mite

 matrix index : 2

 ENTER ELEMENTS OF KEY MATRIX (row by row) :
3
1
5
2

 PLAIN TEXT:mite

 ENCRYPTED TEXT:syjz
 Enter inverse matrix values :2
25
21
3

 DECRYPTED TEXT:mite
```

Practical:-6

AIM: - To Implement Simple Transposition encryption-decryption

CODE:-

```
#include<stdio.h>
#include<string.h>
void cipher(int i,int c);
int findMin();
void makeArray(int,int);
char arr[22][22],darr[22][22],emessage[111],retmessage[111],key[55];
char temp[55],temp2[55];
int k=0;
int main() {
	char *message,*dmessage;
	int i,j,klen,emlen,flag=0;
	int r,c,index,min,rows;
	clrscr();
	printf("Enetr the key\n");
	fflush(stdin);
	gets(key);
	printf("\nEnter message to be ciphered\n");
	fflush(stdin);
	gets(message);
	strcpy(temp,key);
	klen=strlen(key);
	k=0;
	for (i=0; ;i++) {
		if(flag==1)
		   break;
		for (j=0;key[j]!=NULL;j++) {
			if(message[k]==NULL) {
				flag=1;
				arr[i][j]='-';
			} else {
				arr[i][j]=message[k++];
			}
		}
	}
	r=i;
	c=j;
	for (i=0;i<r;i++) {
		for (j=0;j<c;j++) {
			printf("%c ",arr[i][j]);
		}
		printf("\n");
	}
	k=0;
```

```
	for (i=0;i<klen;i++) {
		index=findMin();
		cipher(index,r);
	}
	emessage[k]='\0';
	printf("\nEncrypted message is\n");
	for (i=0;emessage[i]!=NULL;i++)
	  printf("%c",emessage[i]);
	printf("\n\n");
	//deciphering
	emlen=strlen(emessage);
	//emlen is length of encrypted message
	strcpy(temp,key);
	rows=emlen/klen;
	//rows is no of row of the array to made from ciphered message
	rows;
	j=0;
	for (i=0,k=1;emessage[i]!=NULL;i++,k++) {
		//printf("\nEmlen=%d",emlen);
		temp2[j++]=emessage[i];
		if((k%rows)==0) {
			temp2[j]='\0';
			index=findMin();
			makeArray(index,rows);
			j=0;
		}
	}
	printf("\nArray Retrieved is\n");
	k=0;
	for (i=0;i<r;i++) {
		for (j=0;j<c;j++) {
			printf("%c ",darr[i][j]);
			//retrieving message
			retmessage[k++]=darr[i][j];
		}
		printf("\n");
	}
	retmessage[k]='\0';
	printf("\nMessage retrieved is\n");
	for (i=0;retmessage[i]!=NULL;i++)
	   printf("%c",retmessage[i]);
	getch();
	return(0);
}
void cipher(int i,int r) {
	int j;
	for (j=0;j<r;j++) { {
```

```
                        emessage[k++]=arr[j][i];
                }
        }
        // emessage[k]='\0';
}
void makeArray(int col,int row) {
        int i,j;
        for (i=0;i<row;i++) {
                darr[i][col]=temp2[i];
        }
}
int findMin() {
        int i,j,min,index;
        min=temp[0];
        index=0;
        for (j=0;temp[j]!=NULL;j++) {
                if(temp[j]<min) {
                        min=temp[j];
                        index=j;
                }
        }
        temp[index]=123;
        return(index);
}
```

OUTPUT:-

```
Enetr the key
hello

Enter message to be ciphered
how are you
h o w   a
r e   y o
u - - - -

Encrypted message is
oe-hruw - y-ao-
```

```
Array Retrieved is
h o w   a
r e   y o
u - - - -

Message retrieved is
how are you----
```

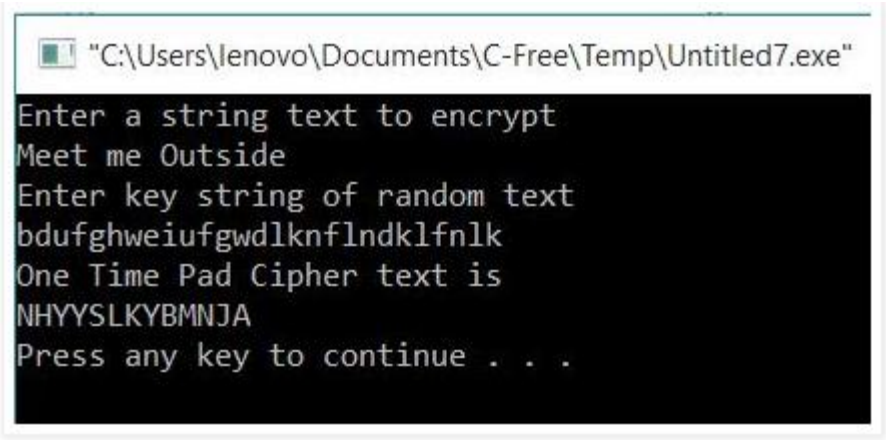
Practical:-7

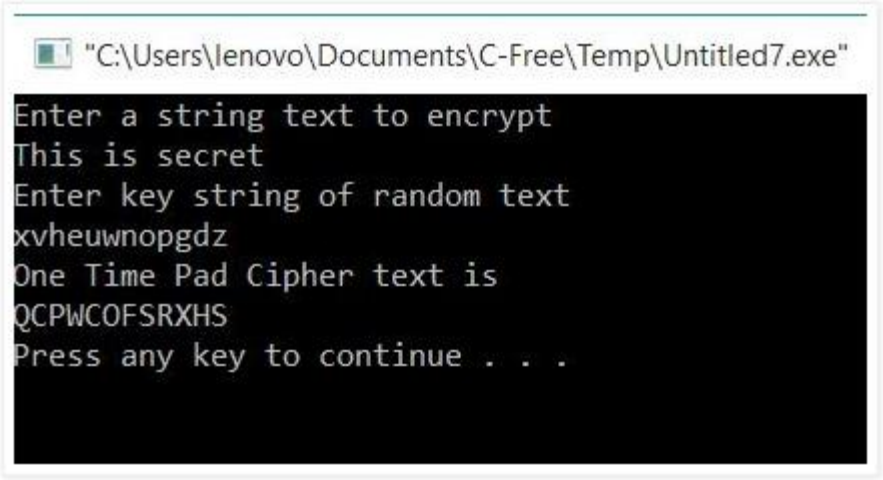
AIM: - Implement one time pad encryption-decryption.

CODE:-

```
#include<stdio.h>
#include<string.h>
#include<ctype.h>
main()
{
 //All the text which ever entered is converted to upper and without spaces
 int i,j,len1,len2,numstr[100],numkey[100],numcipher[100];
 char str[100],key[100],cipher[100];
 printf("Enter a string text to encrypt\n");
 gets(str);
 for(i=0,j=0;i<strlen(str);i++)
 {
  if(str[i]!=' ')
  {
   str[j]=toupper(str[i]);
   j++;
  }
 }
 str[j]='\0';
 //obtaining numerical plain text ex A-0,B-1,C-2
    for(i=0;i<strlen(str);i++)
    {
     numstr[i]=str[i]-'A';
    }
    printf("Enter key string of random text\n");
    gets(key);
 for(i=0,j=0;i<strlen(key);i++)
 {
  if(key[i]!=' ')
  {
   key[j]=toupper(key[i]);
   j++;
  }
 }
 key[j]='\0';
 //obtaining numerical one time pad(OTP) or key
    for(i=0;i<strlen(key);i++)
    {
     numkey[i]=key[i]-'A';
    }

    for(i=0;i<strlen(str);i++)
    {
```

```
 numcipher[i]=numstr[i]+numkey[i];
 }
 //To loop the number within 25 i.e if addition of numstr and numkey is 27 then numcipher should be
1
 for(i=0;i<strlen(str);i++)
 {
  if(numcipher[i]>25)
  {
   numcipher[i]=numcipher[i]-26;
  }
 }
 printf("One Time Pad Cipher text is\n");
 for(i=0;i<strlen(str);i++)
 {
   printf("%c",(numcipher[i]+'A'));
 }
 printf("\n");

}
```





Practical:-8

AIM: - Implement Diffie-Hillmen key exchange method.

Introduction:

- **Diffie–Hellman key exchange** (**D–H**) is a method of securely exchanging cryptographic keys over a public channel and was one of the first public-key protocols. Diffie–Hellman Key Exchange establishes a shared secret between two parties that can be used for secret communication for exchanging data over a public network. The following conceptual diagram illustrates the general idea of the key exchange by using colors instead of very large numbers.
- The simplest and the original implementation of the protocol uses the multiplicative group of integers modulo p, where p is prime, and g is a primitive root modulo p. These two values are chosen in this way to ensure that the resulting shared secret can take on any value from 1 to p–1. Here is an example of the protocol, with non-secret values in blue, and secret values in **red**.

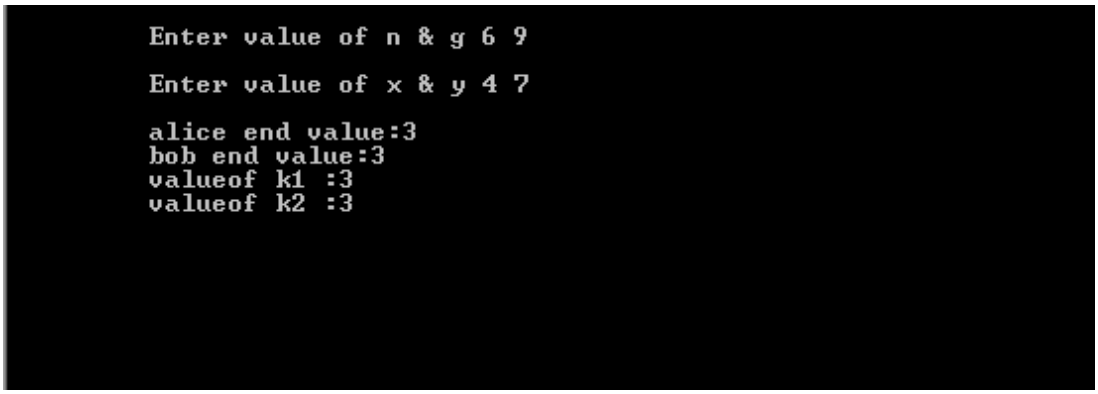
1. Alice and Bob agree to use a modulus $p = 23$ and base $g = 5$ (which is a primitive root modulo 23).
2. Alice chooses a secret integer $\boldsymbol{a} = \mathbf{6}$, then sends Bob $A = g^a \bmod p$
   - $A = 5^6 \bmod 23 = 8$
3. Bob chooses a secret integer $\boldsymbol{b} = \mathbf{15}$, then sends Alice $B = g^b \bmod p$
   - $B = 5^{15} \bmod 23 = 19$
4. Alice computes $\boldsymbol{s} = B^a \bmod p$
   - $\boldsymbol{s} = 19^6 \bmod 23 = \mathbf{2}$
5. Bob computes $\boldsymbol{s} = A^b \bmod p$
   - $\boldsymbol{s} = 8^{15} \bmod 23 = \mathbf{2}$
6. Alice and Bob now share a secret (the number **2**).

Both Alice and Bob have arrived at the same value s.

CODE:-

```
#include<conio.h>
#include<iostream>
#include<math.h>
using namespace std;
int alice(int,int,int);
int bob(int,int,int);

int main()
{
 long int g,x,y,a,b,k1,k2,n;

 cout<<"\n\t Enter value of n & g";
 cin>>n>>g;

 cout<<"\n\t Enter value of x & y";
 cin>>x>>y;

 a=alice(n,g,x);
 cout<<"\n\t alice end value:"<<a;

 b=bob(n,g,y);
 cout<<"\n\t bob end value:"<<b;

 k1=alice(n,b,x);
 cout<<"\n\t valueof k1 :"<<k1;

 k2=alice(n,a,y);
 cout<<"\n\t valueof k2 :"<<k2;
 getch();
}

int alice(int n,int g,int x)
{
 long int a,a1;
 a1=pow(g,x);
 a=a1%n;
 return(a);
}
int bob(int n,int g,int y)
{
 long int b,b1,k2,t2;
 b1=pow(g,y);
 b=b1%n;
 return(b);
}
```

OUTPUT:-


Enter value of n & g 6 9
Enter value of x & y 4 7
alice end value:3
bob end value:3
valueof k1 :3
valueof k2 :3

Practical:-9

AIM: - Implement RSA encryption-decryption algorithm.

Introduction:

- RSA is one of the first practical public-key cryptosystems and is widely used for secure data transmission.
- In such a cryptosystem, the encryption key is public and it is different from the decryption key which is kept secret (private).
- In the RSA, this asymmetry is based on the practical difficulty of the factorization of the product of two large prime numbers, the "factoring problem".

CODE:-

```
#include<stdio.h>
#include<math.h>
int gcd(int a, int b) // Returns gcd of a and b
{
   if (a == 0)
         return b;
   return gcd(b%a, a);
}
int main()
{
   // Two random prime numbers
   double p = 3;
   double q = 7;
   double d, msg, c, m;
   int k;
   // First part of public key:
   double n = p*q;

   // Finding other part of public key.
   // e stands for encrypt
   double e = 2;
   double phi = (p-1)*(q-1);
   while (e < phi)
   {
         // e must be co-prime to phi and
         // smaller than phi.
         if (gcd(e, phi)==1)
            break;
         else
```

```
            e++;
    }
    // Private key (d stands for decrypt)
      k = 2; // A constant value
      d = (1 + (k*phi))/e;

    // Message to be encrypted
    msg = 12;
    printf("Message data = %lf",msg);

    // Encryption c = (msg ^ e) % n
    c = pow(msg, e);
    c = fmod(c, n);
    printf("\nEncrypted data = %lf", c);

    // Decryption m = (c ^ d) % n
    m = pow(c, d);
    m = fmod(m, n);
    printf("\nOriginal Message Sent = %lf",m);

    return 0;
}
```

OUTPUT:-

```
Message data = 12.000000
Encrypted data = 3.000000
Original Message Sent = 12.000000
--------------------------------
Process exited after 0.0316 seconds with return value 0
Press any key to continue . . .
```

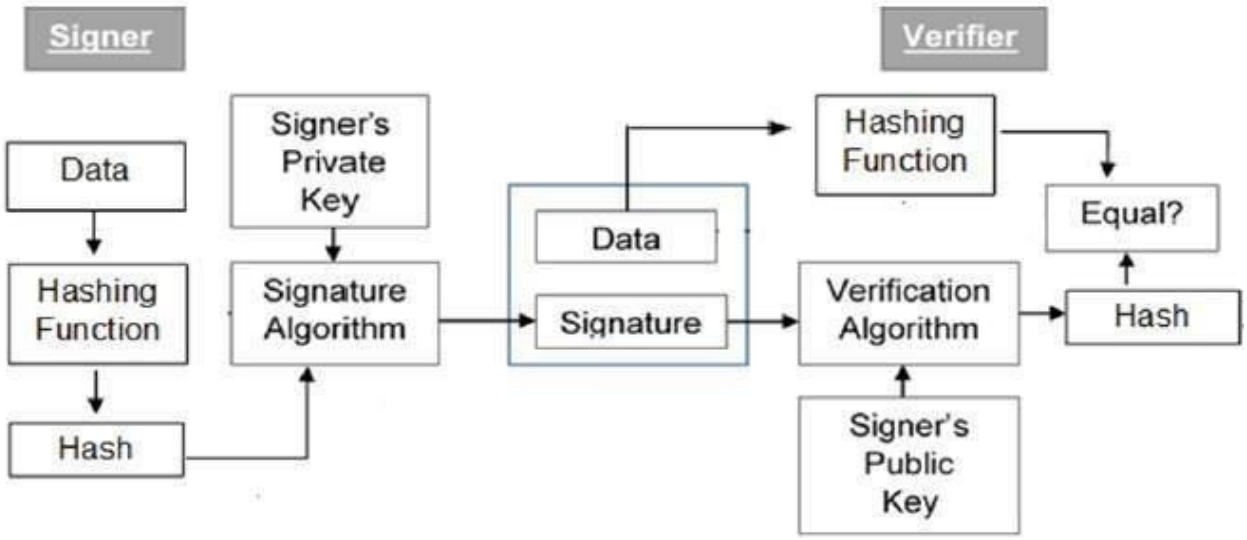
Practical:-10

AIM: - Demonstrate working of Digital Signature using Cryptool

Description: A digital signature is a technique that binds a person/entity to the digital data. This binding can be independently verified by receiver as well as any third party.

Digital signature is a cryptographic value that is calculated from the data and a secret key known only by the signer.

The model of digital signature scheme is depicted in the following illustration − The model of digital signature scheme is depicted in the following illustration –



The following points explain the entire process in detail −

Each person adopting this scheme has a public-private key pair.

Generally, the key pairs used for encryption/decryption and signing/verifying are different. The private key used for signing is referred to as the signature key and the public key as the verification key.

Signer feeds data to the hash function and generates hash of data.

Hash value and signature key are then fed to the signature algorithm which produces the digital signature on given hash. Signature is appended to the data and then both are sent to the verifier.

Verifier feeds the digital signature and the verification key into the verification algorithm. The verification algorithm gives some value as output.

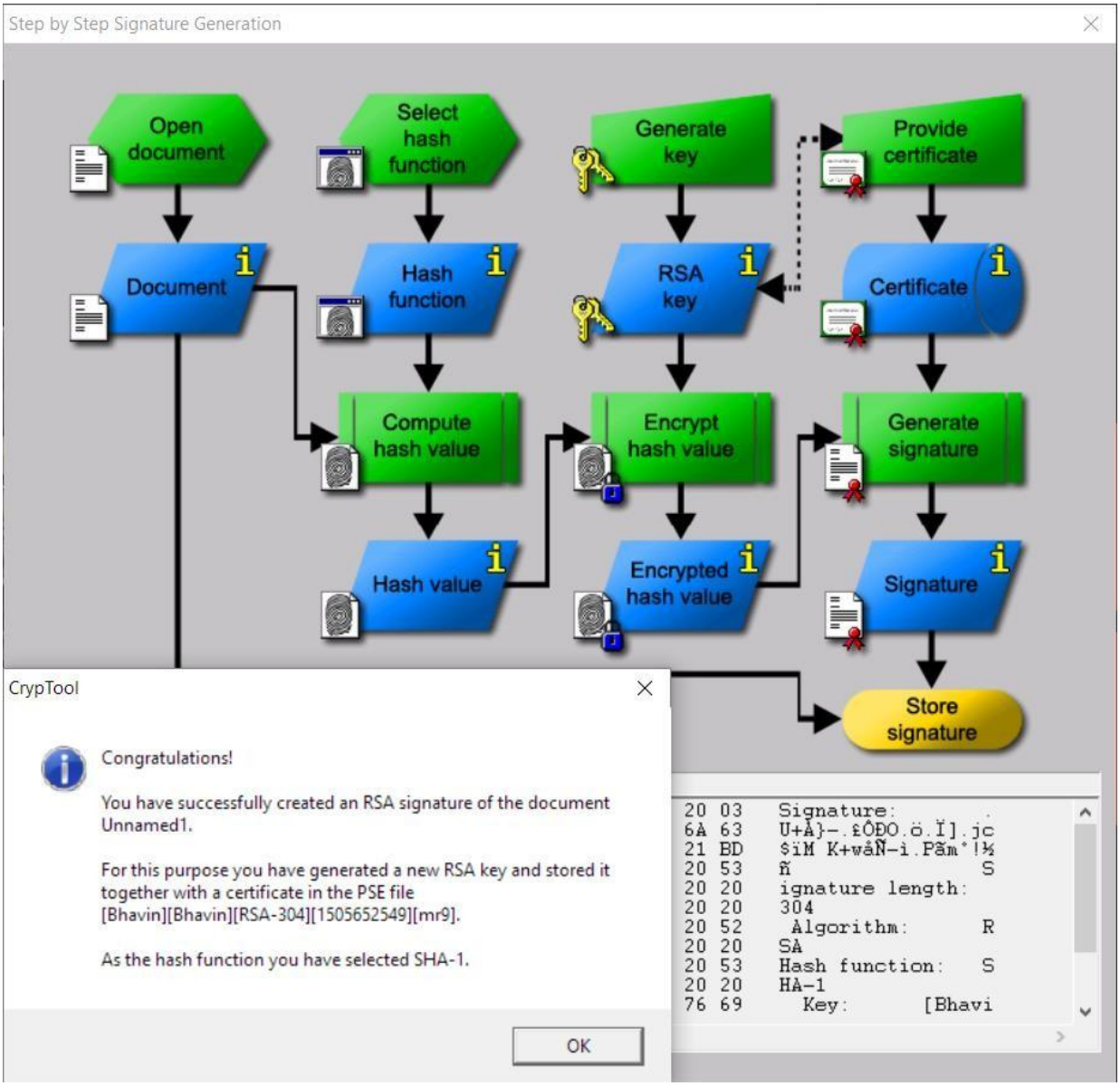
Verifier also runs same hash function on received data to generate hash value.

For verification, this hash value and output of verification algorithm are compared. Based on the comparison result, verifier decides whether the digital signature is valid.

Since digital signature is created by ‘private’ key of signer and no one else can have this key; the signer cannot repudiate signing the data in future.

Screenshot:

Implementation in Cryptool



Practical:-11

AIM:- Perform various encryption-decryption techniques with cryptool.

Description:

CrypTool is an open source project. Main result is the free e-learning software CrypTool illustrating cryptographic and cryptanalytic concepts.

According to "Hakin9"CrypTool is worldwide the most widespread e-learning software in the field of cryptology.

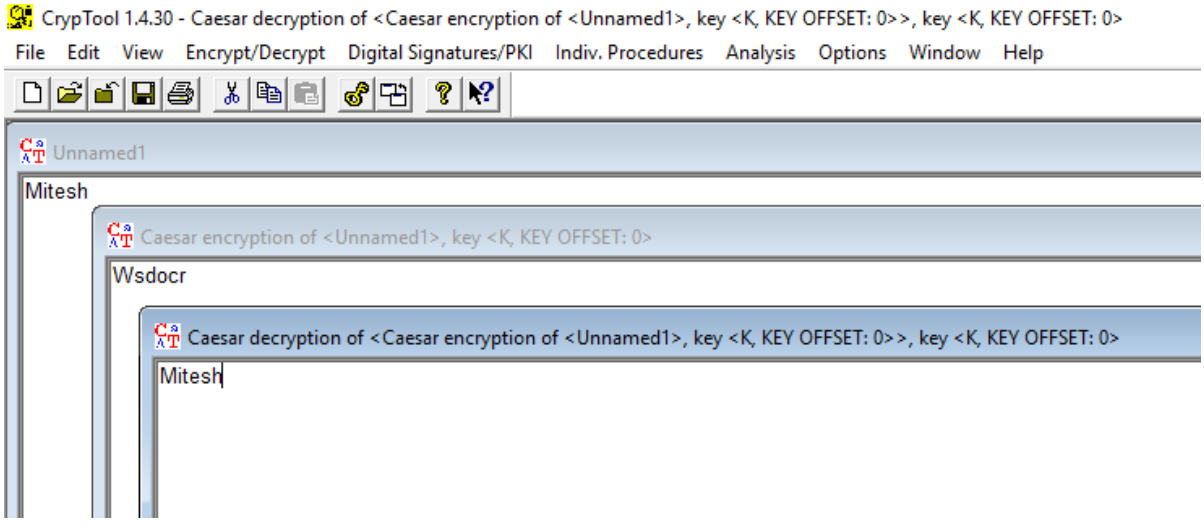
CrypTool implements more than 300 algorithms. Users can adjust these with own parameters. The graphical interface, online documentation, analytic tools and algorithms of CrypTool introduce users to the field of cryptography.

CrypTool contains the most classical ciphers as well as modern symmetric and asymmetric cryptography including RSA, ECC, digital signatures, hybrid encryption, homomorphic encryption, and Diffie–Hellman key exchange.

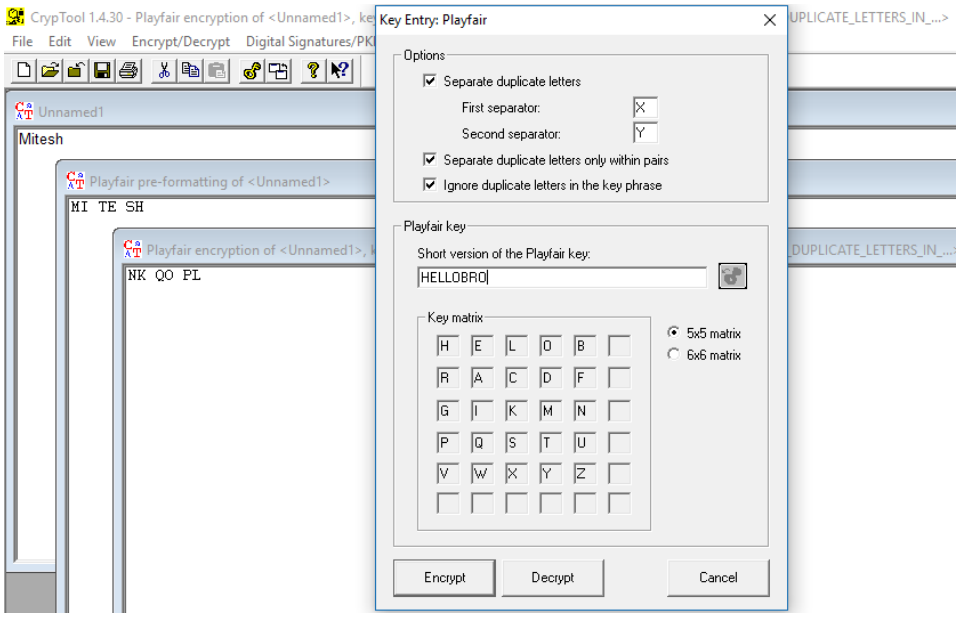
Also methods from the area of quantum cryptography (like BB84 key exchange protocol) and the area of post-quantum cryptography (like McEliece, WOTS, Merkle-Signature-Scheme, XMSS) are implemented. Many methods (for instance Huffman code, AES, Keccak, MSS) are visualized.

Screenshot:

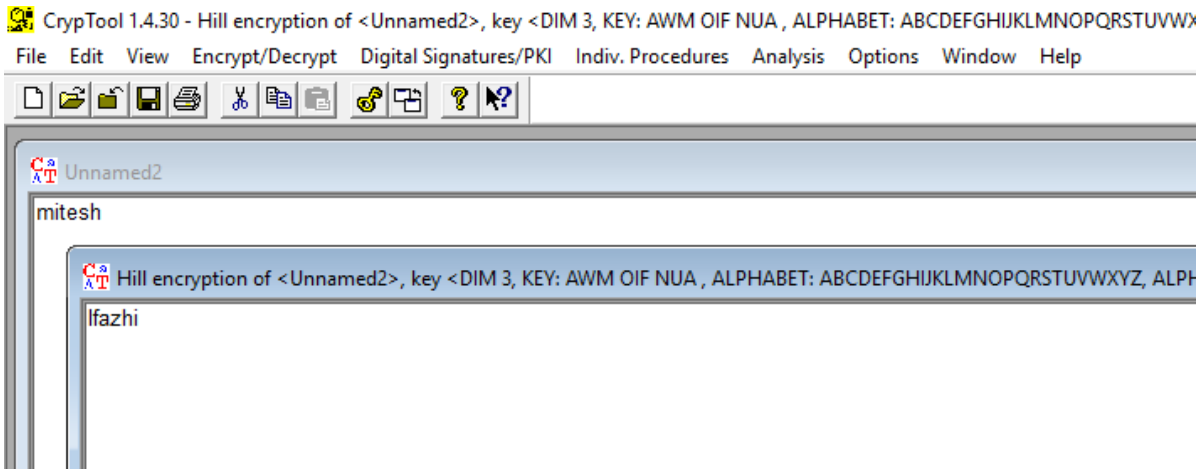
CAESER DEMONSTRATION IN CRYPTOOL



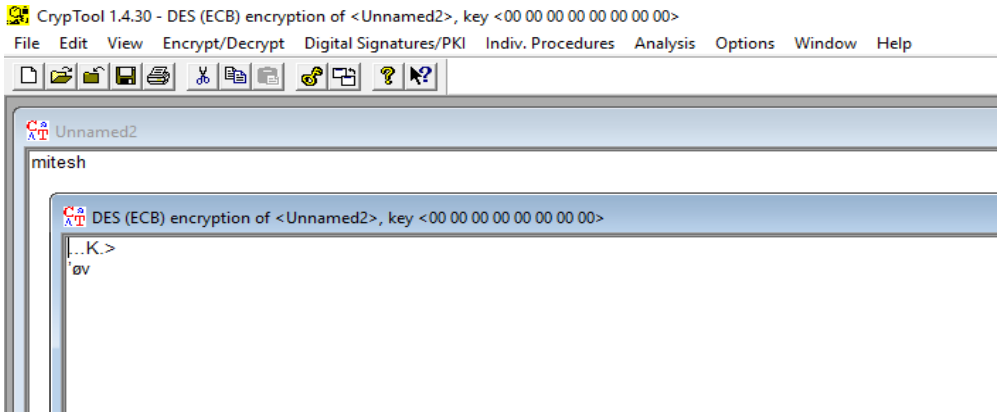
PLAYFAIR DEMONSTRATION IN CRYPTOOL



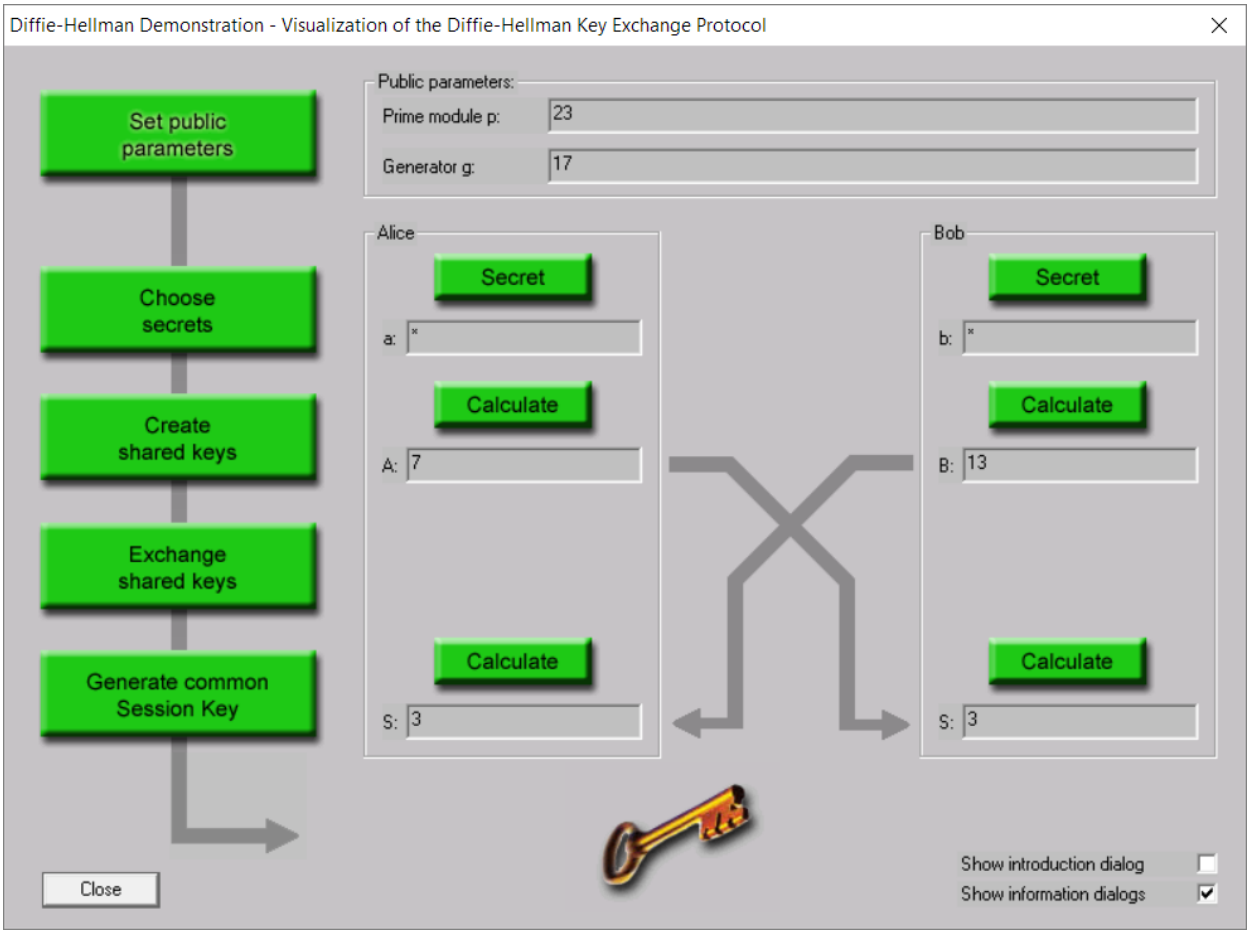
HILL CIPHER DEMONSTRATION IN CRYPTOOL



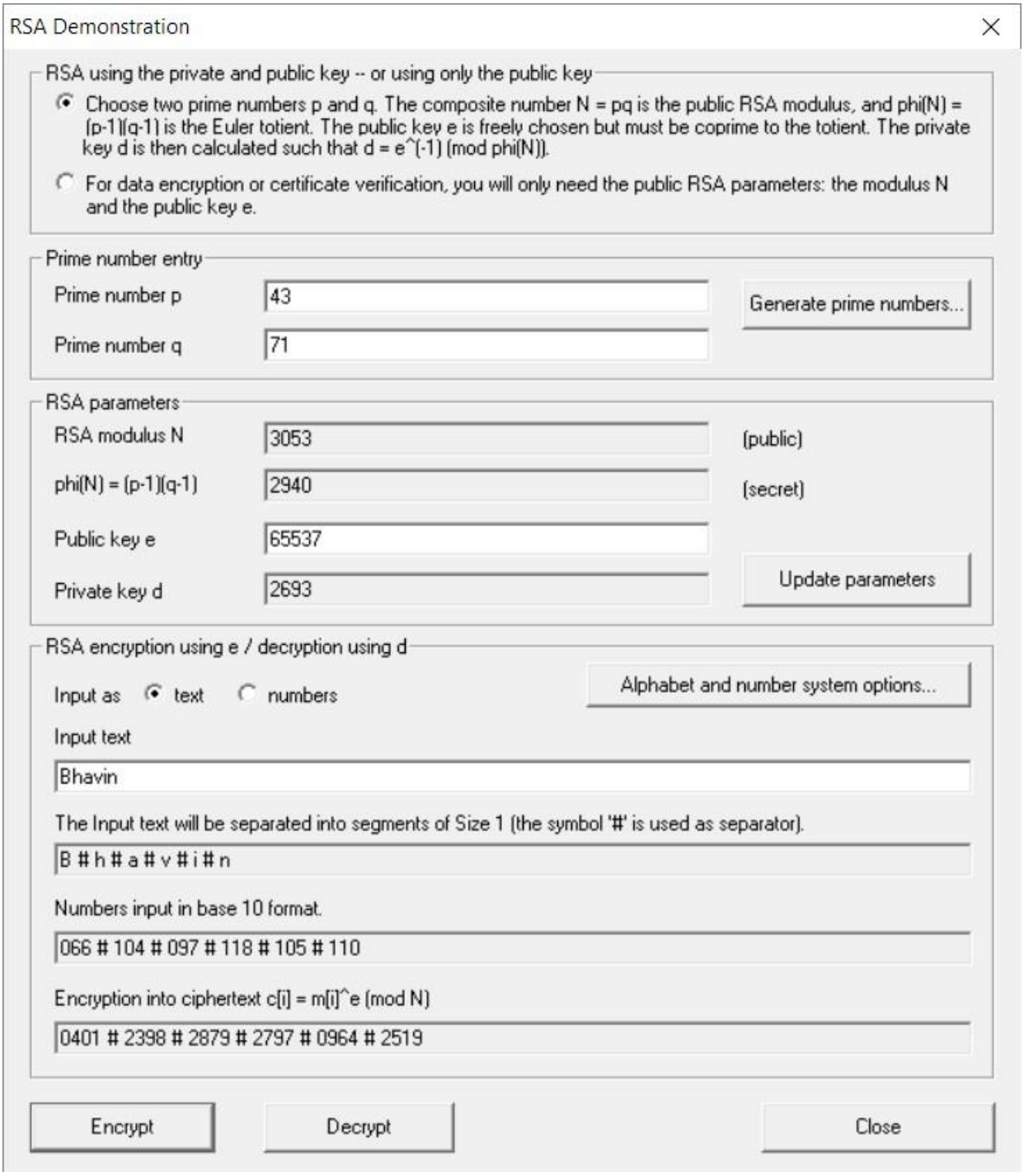
DES DEMONSTRATION IN CRYPTOOL



DIFFI-HELLMEN KEY EXCHANGE METHOD IN CRYPTOOL



RSA DEMONSTRATION IN CRYPTOOL



```
⊟ Address Resolution Protocol (request)
    Hardware type: Ethernet (1)
    Protocol type: IP (0x0800)
    Hardware size: 6
    Protocol size: 4
    Opcode: request (1)
    Sender MAC address: 10.0.62.43 (10:60:4b:90:43:2d)
    Sender IP address: 10.0.62.43 (10.0.62.43)
    Target MAC address: 00:00:00_00:00:00 (00:00:00:00:00:00)
    Target IP address: 10.0.62.254 (10.0.62.254)
```

```
0000  ff ff ff ff ff ff 10 60  4b 90 43 2d 08 06 00 01   .......` K.C-....
0010  08 00 06 04 00 01 10 60  4b 90 43 2d 0a 00 3e 2b   .......` K.C-..>+
0020  00 00 00 00 00 00 0a 00  3e fe 00 00 00 00 00 00   ........ >.......
0030  00 00 00 00 00 00 00 00  00 00 00 00              ........ ....
```